



UNIVERSIDADE FEDERAL DA PARAÍBA
CENTRO DE INFORMÁTICA
CURSO DE ENGENHARIA DE COMPUTAÇÃO

Aluno:

GABRIEL ASSIS GUEDES ROSAS - 20180029497

RELATÓRIO PROJETO FINAL

JOÃO PESSOA
2026

1 - Introdução:

Este relatório descreve as decisões de arquitetura e implementação adotadas no desenvolvimento do Sistema de Gestão Hospitalar para a disciplina de Banco de Dados. O projeto integra um banco de dados relacional PostgreSQL a uma aplicação Java em camada com Spring Boot e CLI interativo, contemplando modelagem relacional, ORM, rotinas em PL/pgSQL (procedures, triggers e views) e manipulação de dados semiestruturados em JSONB.

2 - Decisões de Implementação e Arquitetura:

2.1 - Escolha do Framework ORM (Spring Data JPA / Hibernate):

A escolha do Spring Data JPA com Hibernate como solução de ORM (Object-Relational Mapping) baseou-se nos seguintes aspectos:

- **Experiência Prévia e Produtividade:** A breve experiência anterior com Java e Spring Boot em projetos acadêmicos e o uso das mesmas tecnologias em estágios acelerou o tempo de desenvolvimento e a curva de aprendizado na integração com o banco de dados.
- **Mapeamento Transparente:** Permite o mapeamento direto de entidades relacionais (Paciente, Atendimento, Preceptor, Residente, Unidade de Saúde) para objetos Java, abstraindo operações repetitivas de CRUD.
- **Suporte a Queries Híbridas:** Facilidade de alternar entre JPQL (Java Persistence Query Language) para consultas orientadas a objetos e Native Queries SQL para execução direta de rotinas avançadas e Views.
- **Gestão Automática de Transações:** O gerenciamento de transações (@Transactional) garante a integridade ACID nas chamadas do banco.

2.2 - Arquitetura de Regras de Negócio: Triggers vs. Stored Procedures:

Para otimizar o desempenho e manter a integridade dos dados, adotou-se uma divisão clara entre o processamento procedural no banco de dados e as chamadas da aplicação:

a) **Uso de Stored Procedures (PL/pgSQL):** As Stored Procedures foram utilizadas para operações complexas iniciadas explicitamente pela aplicação CLI que envolvem múltiplas transações ou encadeamento de ações.

- **Exemplo (sp_registrar_atendimento_completo):** Responsável por validar regras de negócio antes de registrar um novo atendimento (como verificação de choque de horário do paciente e disponibilidade da equipe médica na unidade). A escolha por Procedure centralizou a lógica transacional no banco, reduzindo o tráfego de rede entre a aplicação e o PostgreSQL.

b) Uso de Triggers (Gatilhos Automáticos): As Triggers foram empregadas estritamente para automações reativas e garantia de integridade contratual que não devem depender do código da aplicação para serem executadas.

- Exemplo (trg_check_sobreposicao_escala): Acionada no banco para garantir imperativamente que nenhum paciente possua dois atendimentos simultâneos.
- Exemplo (trg_audita_atendimento): Acionada para registrar alterações em tabelas de auditoria de forma totalmente transparente para o código Java..

2.3 - Estrutura JSONB e Views Analíticas:

Dados Semiestruturados (JSONB): O campo 'dados_medicos' da tabela Atendimento foi modelado utilizando o tipo primitivo JSONB do PostgreSQL. Isso permitiu flexibilidade para armazenar diagnósticos, receitas e exames sem a necessidade de esquemas rígidos ou alterações de DDL a cada novo parâmetro médico.

- Views Analíticas: Foram criadas Views nativas (como vw_pacientes_internados e vw_estatisticas_atendimentos_mensal) para sumarizar dados complexos. A aplicação consome essas Views via Native Query no Spring Data JPA, isolando a complexidade do SQL analítico da camada de apresentação da CLI.

3 - Conclusão:

O desenvolvimento do sistema permitiu consolidar na prática os conceitos de persistência com ORM e rotinas em PL/pgSQL. A separação entre Triggers para automação reativa e Procedures para fluxos mostrou-se eficiente, garantindo integridade de dados e alta performance da aplicação.